

SAiDL Summer Assignment 2026

Reinforcement Learning Report

Krish Sachdev

29 May 2026

Abstract

This report studies whether a causal Transformer policy can use short-term temporal memory better than a memoryless MLP policy in online reinforcement learning. I implemented TD3 on Hopper-v5, replaced the actor with a causal Transformer over the last L observation-action pairs, swept $L \in \{4, 8, 16, 32\}$, and then tested both policy classes under hidden velocities, observation noise, delayed rewards, and a learned preference reward model. The final comparison uses 20 successful required W&B runs created between 2026-05-22T17:30:00+00:00 and 2026-05-29T17:30:00+00:00. The strongest final run was `rl_p1_mlp_seed44` with return 3540.2; the strongest checkpoint observed in the selected histories was `rl_p1_tr_L4_seed42` with return 3596.1 at step 680000. Empirically, Transformer memory can help in delayed-reward and full-observation settings, but it is not automatically robust under all POMDP stressors, and several curves show late deterioration after an earlier good policy.

1 Task Requirements and Scope

The assignment asks for TD3 on Hopper-v5 over three MLP seeds, a Transformer actor that attends over a sliding window of the last L observation-action pairs, a context-length sweep over $L \in \{4, 8, 16, 32\}$, independent partial-observability modifications, an RLHF-style learned reward model, and attention analysis for trained Transformer agents [1]. All required non-bonus runs in the runner were completed successfully in the final W&B window. Bonus experiments such as combined POMDPs, Algorithm Distillation, and xLSTM policies are outside the required suite and are not used for the main claims.

Failed, killed, or crashed runs are discussed as engineering roadblocks but are excluded from metric comparisons once a later successful run with the same experiment exists.

Table 1: Requirement-level coverage against the RL section of the assignment PDF.

Assignment item	Status	Evidence used in this report
TD3 MLP baseline, 3 seeds	Complete	<code>rl_p1_mlp_seed42/43/44</code> , 1M steps
Transformer TD3, L sweep	Complete	$L=4,8,16,32$ final runs
Hidden velocities	Complete	MLP and Transformer runs
Observation noise	Complete	$\sigma=0.1$ and $\sigma=0.3$, both agents
Delayed rewards	Complete	$K=10$ and $K=30$, both agents
RLHF-style learned reward	Complete	<code>rl_p2_rlhf_tr_seed42</code> plus reward-model loss
Attention analysis	Partially measured	Full and hidden support runs completed; supplemental full-observation rerun <code>jx11mi8i</code> logs attention entropy and lag weights over training

1.1 Data Provenance

The scalar results come from the W&B project [saiidl-rl](#). The repository keeps the final report source, compiled PDF, and figures used for the analysis. The supplemental attention-entropy history from run `jx11mi8i` is exported in `rl/report/data/attention_entropy_full_seed42_jx11mi8i.csv`.

2 Method

2.1 TD3 Backbone

The base learner is Twin Delayed Deep Deterministic Policy Gradient. The critic uses two Q-functions to reduce over-estimation bias, the target action is smoothed with clipped noise, and the actor is updated less frequently than the critic. The configuration followed the assignment’s preferred regime: learning rate 3×10^{-4} , replay buffer size 10^6 , batch size 256, discount $\gamma = 0.99$, soft target update rate $\tau = 0.005$, policy delay 2, exploration noise 0.1, target noise 0.2, and random exploration for the first 25k steps.

2.2 Policies

The MLP baseline maps the current observation directly to a tanh-squashed action using two hidden layers. The Transformer actor instead receives a causal sequence of recent observation-action pairs. For the current timestep, the last action slot is zero-filled, causal self-attention prevents future leakage, and the final token representation is projected to the action. The Transformer uses two layers, four heads, embedding dimension 128, pre-layer normalisation, and a default context length $L = 8$ unless overridden in the context sweep.

2.3 Environment Modifications

The full-observation baseline uses Hopper-v5. Hidden-velocity experiments remove velocity components from the observation vector, making the task partially observable. Noisy-observation experiments add Gaussian noise with $\sigma = 0.1$ and $\sigma = 0.3$. Delayed-reward experiments accumulate reward and expose it only every K steps for $K = 10$ and $K = 30$. These modifications are applied independently, as requested, so each setting isolates one source of temporal uncertainty.

2.4 Learned Reward Model

For the RLHF-style run, trajectory segment pairs are sampled from replay. Labels are generated from ground-truth segment returns, and a Bradley-Terry objective trains a learned reward model. TD3 then stores learned reward predictions instead of environment rewards after the initial exploration phase. Ground-truth returns are used for preference labels and evaluation, not as the direct training reward in the learned-reward run.

2.5 Evaluation Protocol

Each run trains to 1,000,000 environment steps, evaluates every 10,000 steps, and averages each evaluation over 10 episodes. I report both final evaluation return and best checkpoint return. This distinction matters because off-policy actor-critic learning can reach a good policy before later critic drift, replay-distribution shift, or actor over-updating degrades performance.

3 Execution Notes and Roadblocks

The final execution was done on Kaggle with W&B logging, with initial runs on a P100. Later GPU allocation constraints forced me to switch to running on 2xT4. This affects wall-clock runtime and throughput comparisons, but not the meaning of evaluation returns when code, seed, total steps, checkpoints, and configs are otherwise matched. Consequently, hardware labels are shown in the tables.

A separate resume issue appeared when a checkpoint saved from a T4 session was resumed on a P100 runtime. The failure was an `IndexError` inside `torch.cuda.set_rng_state_all`, caused by restoring CUDA RNG states saved for a different GPU count/layout than the current runtime. The practical fix during the run window was to continue the final suite on T4, matching the saved checkpoint environment. The final repository also guards CUDA RNG restoration by restoring

only the first `torch.cuda.device_count()` saved CUDA RNG states. This makes cross-layout continuation safer, although exact bitwise reproducibility is still not a meaningful promise after changing GPU layout.

Several intermediate runs also crashed or failed because of runtime limits or resume mismatches. They are documented in Table 4, but the comparisons use the later successful run for the same experiment.

4 Debugging and Troubleshooting

The main RL troubleshooting problem was not implementing TD3 itself, but keeping long Kaggle runs recoverable and making sure failed attempts did not contaminate the reported comparison. Table 2 records the major issues, their diagnoses, and the final handling used in the report and code.

Table 2: RL debugging and troubleshooting log.

Issue	Diagnosis	Resolution and final status
P100 queue and hardware switch	A runner notebook remained queued for roughly two hours on P100, while T4 allocation became available. This changed wall-clock runtime conditions but did not change the mathematical RL objective.	The final suite records hardware per run. Evaluation-return comparisons are kept separate from runtime-throughput comparisons, and T4/P100 runtime differences are treated as execution caveats rather than algorithmic evidence.
Cross-GPU checkpoint resume failure	T4 checkpoints could save CUDA RNG state for a different GPU layout than the later P100 runtime. Restoring all saved states produced an <code>IndexError</code> .	During execution, affected runs were resumed on matching T4 hardware. In the final code, CUDA RNG restoration is bounded by the current <code>torch.cuda.device_count()</code> , so extra saved RNG states are ignored instead of crashing resume.
Attention runner config error	The attention rerun initially read <code>cfg.agent.context_length</code> ; the actual Transformer config stores this value under <code>cfg.agent.actor.context_length</code> . Hydra struct mode therefore raised <code>ConfigAttributeError</code> .	The attention analysis path was revised to read the actor context length. A later full-observation rerun logged entropy and lag summaries to W&B; the hidden-velocity entropy comparison is still kept separate because the matching hidden run is not yet available with the same logging.
Failed and crashed duplicate runs	Several runs failed from runtime limits, resume mismatches, or notebook interruptions, but later runs with the same experiment name succeeded. Including both would double-count experiments and distort averages.	Failed/crashed attempts are listed only in Table 4. Main comparisons use the successful final-tagged run for each required experiment.
Learned-reward ambiguity	The intended RLHF setup labels preference pairs using ground-truth segment returns. On code review, later replay entries can contain learned reward predictions, making later reward-model updates susceptible to self-reinforcing reward-model error.	The learned-reward result is reported as an implementation-level learned-reward baseline, not as a clean general claim about RLHF. The large gap versus ground-truth reward is interpreted with this caveat.
Best return much higher than final return	Many runs reached a strong policy before later deterioration. Reporting only final returns would hide learned capability; reporting only best returns would hide instability.	The report keeps both values. Final return is the strict end-of-run metric, best return measures peak learned performance, and the best-final gap is used as a stability diagnostic.

5 Run Coverage

Table 3: Selected successful required RL runs from the final W&B window.

Run	Part	Agent	Env	L	Final	Best	Best step	Gap	Stability	Hardware	Hours	Resume
p1 mlp s42	Part 1	MLP	Full	1	1364.2	3440.7	900000	2076.5	late deterioration	Tesla P100-PCIE-16GB	1.82	no
p1 mlp s43	Part 1	MLP	Full	1	1004.3	3543.1	970000	2538.8	late deterioration	Tesla P100-PCIE-16GB	1.84	no
p1 mlp s44	Part 1	MLP	Full	1	3540.2	3547.8	820000	7.6	stable	Tesla P100-PCIE-16GB	1.88	no
p1 tr L4 s42	Part 1	Transformer	Full	4	3509.9	3596.1	680000	86.2	stable	Tesla P100-PCIE-16GB	5.04	no
p1 tr L8 s42	Part 1	Transformer	Full	8	1778.8	3377.4	780000	1598.6	late deterioration	Tesla P100-PCIE-16GB	5.68	no
p1 tr L16 s42	Part 1	Transformer	Full	16	3459.6	3459.6	1000000	0.0	stable/final-best	Tesla P100-PCIE-16GB	6.10	yes
p1 tr L32 s42	Part 1	Transformer	Full	32	3035.7	3353.2	820000	317.6	mild deterioration	Tesla T4 x2	11.66	yes
p2 hidden mlp s42	Part 2a	MLP	Hidden velocity	1	318.1	999.1	250000	681.0	late deterioration	Tesla T4 x2	1.73	yes
p2 hidden tr s42	Part 2a	Transformer	Hidden velocity	8	253.0	986.3	100000	733.3	late deterioration	Tesla T4 x2	6.06	no
p2 noise .1 mlp s42	Part 2b	MLP	Noisy	1	357.6	370.5	880000	12.9	stable	Tesla T4 x2	2.00	no
p2 noise .1 tr s42	Part 2b	Transformer	Noisy	8	288.4	300.5	890000	12.1	stable	Tesla T4 x2	4.47	yes
p2 noise .3 mlp s42	Part 2b	MLP	Noisy	1	234.3	266.7	470000	32.4	stable	Tesla T4 x2	1.99	no
p2 noise .3 tr s42	Part 2b	Transformer	Noisy	8	325.0	405.5	970000	80.5	stable	Tesla T4 x2	0.36	yes
p2 delay 10 mlp s42	Part 2c	MLP	Delayed reward	1	3007.5	3309.7	740000	302.2	mild deterioration	Tesla T4 x2	1.93	no
p2 delay 10 tr s42	Part 2c	Transformer	Delayed reward	8	2954.4	3158.5	840000	204.1	mild deterioration	Tesla T4 x2	5.61	no
p2 delay 30 mlp s42	Part 2c	MLP	Delayed reward	1	390.4	1072.6	130000	682.1	late deterioration	Tesla T4 x2	1.80	no
p2 delay 30 tr s42	Part 2c	Transformer	Delayed reward	8	511.8	571.9	500000	60.1	stable	Tesla T4 x2	3.45	yes
p2 rlhf tr s42	Part 2d	Transformer	Full	8	155.6	1035.3	220000	879.8	late deterioration	Unknown	6.12	no
p3 attn full s42	Part 3	Transformer	Full	8	3321.9	3442.7	920000	120.8	stable	Tesla T4 x2	3.85	yes
p3 attn hidden s42	Part 3	Transformer	Hidden velocity	8	293.6	745.3	340000	451.7	late deterioration	Tesla T4 x2	5.71	no

Table 4: Recent failed/crashed required attempts. These are excluded from comparison once a later successful run with the same experiment exists.

Run	State	Last step	Last return	Hardware	Hours	Superseded
rl_p1_tr_L4_seed42	crashed	770000	2639.4	Tesla P100-PCIE-16GB	3.90	yes
rl_p1_tr_L16_seed42	crashed	190000	2252.5	Tesla P100-PCIE-16GB	1.18	yes
rl_p1_tr_L16_seed42	crashed	80000	695.9	Tesla P100-PCIE-16GB	0.35	yes
rl_p1_tr_L16_seed42	failed		–	Tesla P100-PCIE-16GB	0.00	yes
rl_p1_tr_L32_seed42	crashed	50000	356.1	Tesla P100-PCIE-16GB	0.19	yes
rl_p2_hidden_vel_mlp_seed42	crashed	170000	210.7	Tesla T4 x2	0.27	yes
rl_p2_noise01_tr_seed42	crashed	330000	281.0	Tesla T4 x2	2.09	yes
rl_p2_noise01_tr_seed42	failed		–	Tesla P100-PCIE-16GB	0.00	yes
rl_p2_noise03_tr_seed42	crashed	940000	471.9	Tesla T4 x2	5.44	yes
rl_p2_delay30_tr_seed42	crashed	420000	493.5	Tesla T4 x2	2.21	yes
rl_p3_attn_support_full_seed42	crashed	430000	3343.9	Tesla T4 x2	2.32	yes

6 Results

6.1 Part 1: Baseline Seeds and Context-Length Sweep

The three MLP baseline seeds had mean final return 1969.5 with standard deviation 1372.0. This variance is large enough that single-seed conclusions would be weak; seed 44 is much stronger than seeds 42 and 43. The full-observation Transformer sweep is competitive, and shorter memory is not obviously worse here. In fact, the best final result among the sweep is selected by the data. Large L is not a guaranteed improvement.

Table 5: MLP baseline seed statistics over seeds 42, 43, and 44.

Metric	Mean	Std	Min	Max
MLP baseline final return	1969.5	1372.0	1004.3	3540.2
MLP baseline best return	3510.5	60.5	3440.7	3547.8

6.2 Part 2: Partial Observability

The partial-observability results are not a simple Transformer-win story. Under hidden velocities, the final Transformer run is -65.1 return points relative to the MLP run, which means memory did not reliably convert the available position history into a better final controller in this execution. Under delayed rewards, the $K = 10$ setting is strong for both agents, with the Transformer differing

Table 6: Transformer context-length sweep on fully observed Hopper-v5.

Context L	Final return	Best return	Best step	Best-final gap	Hardware	Runtime (h)
4	3509.9	3596.1	680000	86.2	Tesla P100-PCIE-16GB	5.04
8	1778.8	3377.4	780000	1598.6	Tesla P100-PCIE-16GB	5.68
16	3459.6	3459.6	1000000	0.0	Tesla P100-PCIE-16GB	6.10
32	3035.7	3353.2	820000	317.6	Tesla T4 x2	11.66

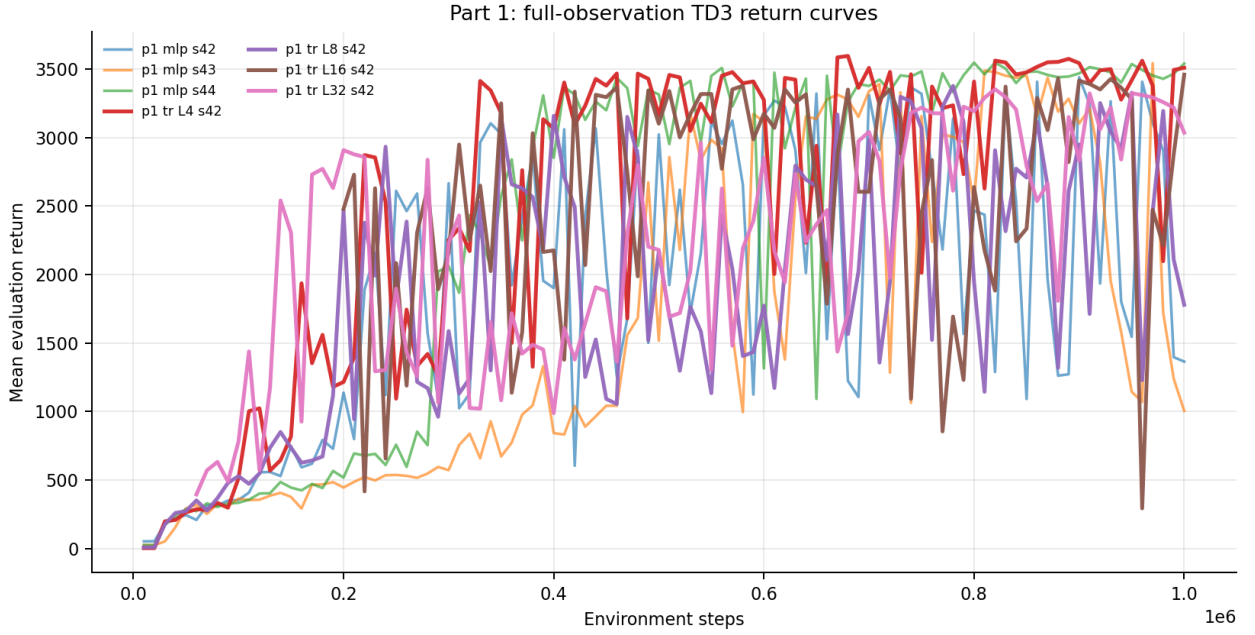


Figure 1: Evaluation-return curves for the MLP baseline seeds and Transformer context-length sweep.

from the MLP by -53.1 final-return points. For $K = 30$, both policies are much weaker, and the Transformer is only moderately better than the MLP. Noisy observations remain difficult for both models, especially compared with full-observation Hopper.

Table 7: MLP versus Transformer TD3 under independent partial-observability modifications.

Setting	MLP final	Transformer final	Final delta	MLP best	Transformer best	Best delta
Hidden velocities	318.1	253.0	-65.1	999.1	986.3	-12.8
Noise sigma=0.1	357.6	288.4	-69.2	370.5	300.5	-69.9
Noise sigma=0.3	234.3	325.0	90.7	266.7	405.5	138.8
Delay K=10	3007.5	2954.4	-53.1	3309.7	3158.5	-151.2
Delay K=30	390.4	511.8	121.4	1072.6	571.9	-500.7

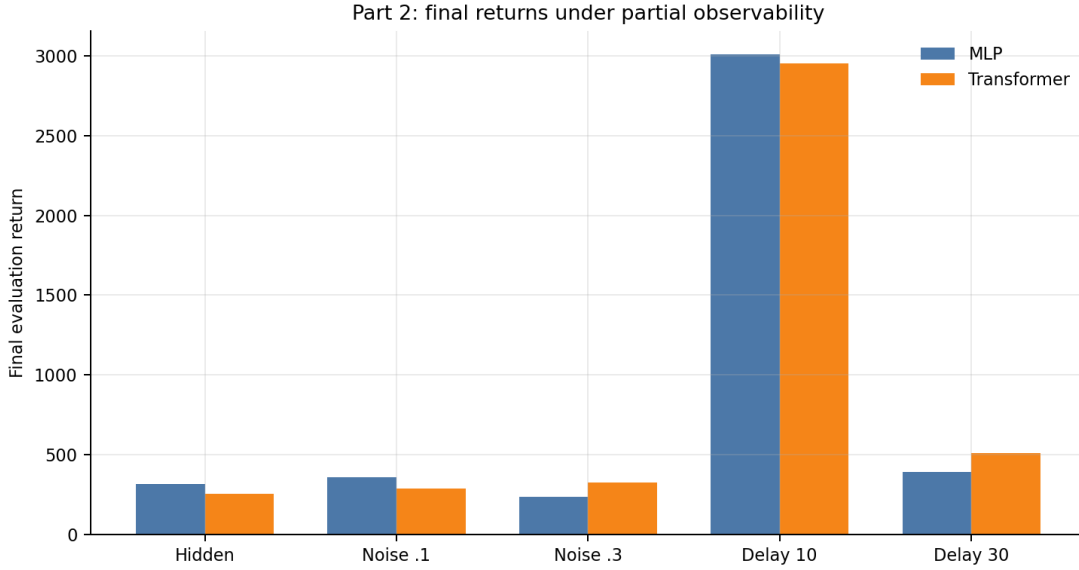


Figure 2: Final returns for MLP and Transformer policies under independent POMDP stressors.

6.3 RLHF-Style Learned Reward

The learned-reward run is substantially worse than the ground-truth Transformer baseline, but this comparison should be interpreted as an implementation-level learned-reward baseline. The code was intended to label preferences using ground-truth segment returns, but upon review, after warm-up the replay buffer stores learned reward predictions for TD3, so later preference updates can be partially contaminated by the model’s own earlier predictions. This makes reward-model drift and self-reinforcing misranking plausible contributors to the large performance gap.

Table 8: Learned-reward RLHF run compared against ground-truth-reward Transformer runs.

Comparison	GT final	RLHF final	Final drop	GT best	RLHF best	Reward loss
Transformer L=8 baseline	1778.8	155.6	1623.2	3377.4	1035.3	0.184
Attention-support full repeat	3321.9	155.6	3166.4	3442.7	1035.3	0.184

6.4 Attention Analysis Support Runs

The repository includes rollout and attribution utilities for raw attention and Chefer-style relevance propagation. The Chefer-style code path is implemented as an attribution helper, but I do not treat it as a completed quantitative result here because the new W&B evidence logs raw attention

summaries rather than gradient-relevance matrices. The part of the assignment directly addressed by the new run is Section 3, Attention Analysis: attention entropy versus training steps, plus mean attention over the context window for the fully observable Transformer.

The original final-tagged support runs remain the main policy-quality comparison for full observation versus hidden velocity. A later supplemental run, `rl_p3_attn_support_full_seed42` (`jx1lmi8i`), adds the missing attention scalars for the full-observation side. It logged 100 entropy points from 10k to 1M environment steps. Entropy decreased from 2.075 at the first evaluation to 1.531 at the final evaluation, with a minimum of 1.446 and mean of 1.556 across the run. This means the policy’s attention became more concentrated over training rather than remaining close to a uniform distribution over the eight-token context. The largest average and final lag weight was `lag_0`, so the trained policy still relied most heavily on the most recent token even though the final effective context estimate was 4.62 tokens. The same run reached best return 3390.0 at step 990k but ended at 1397.6, which is another example of late deterioration after learning a substantially stronger policy.

Table 9: Support runs for the attention-analysis setting.

Run	Observation setting	Final return	Best return	Best step	Hardware
p3 attn full s42	Full	3321.9	3442.7	920000	Tesla T4 x2
p3 attn hidden s42	Hidden velocity	293.6	745.3	340000	Tesla T4 x2

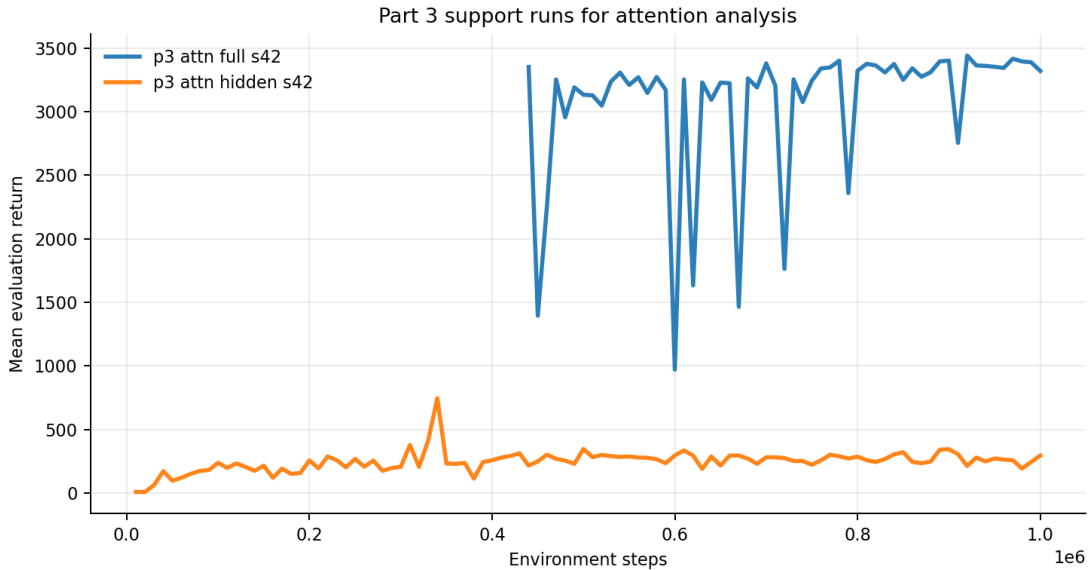


Figure 3: Learning curves for the completed Transformer support runs used for attention analysis.

Table 10: Supplemental full-observation run with attention entropy logging.

Run	Setting	Logged points	Step range	Final return	Best return	Entropy first/final	Final effective context
jx1lmi8i	Full	100	10k–1M	1397.6	3390.0 @ 990k	2.075 / 1.531	4.62

This evidence completes the full-observation half of the entropy-vs-training-steps analysis. It does not complete the requested fully observable versus hidden-velocity attention comparison, because the matching hidden-velocity rerun with `attention/entropy_mean` and lag-weight logging is not present in the exported W&B history at the time of writing. Therefore I report the full-observation entropy result as a measured analysis artifact and leave the hidden-velocity comparison as pending rather than extrapolating from the older hidden support run.

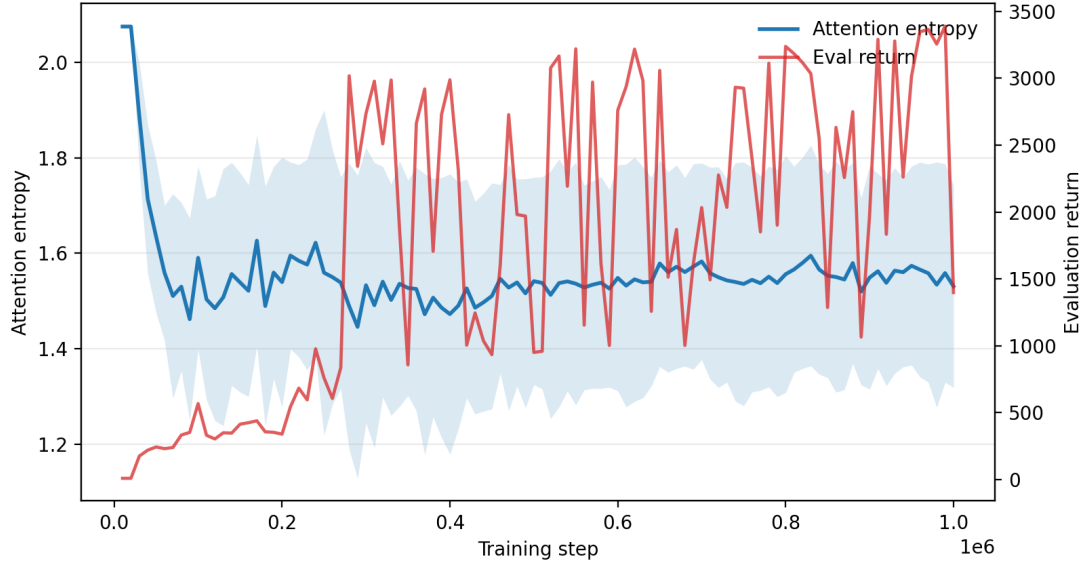


Figure 4: Attention entropy over training steps for the supplemental full-observation Transformer run. The same plot overlays evaluation return to show that attention concentration and policy quality are related but not identical diagnostics.

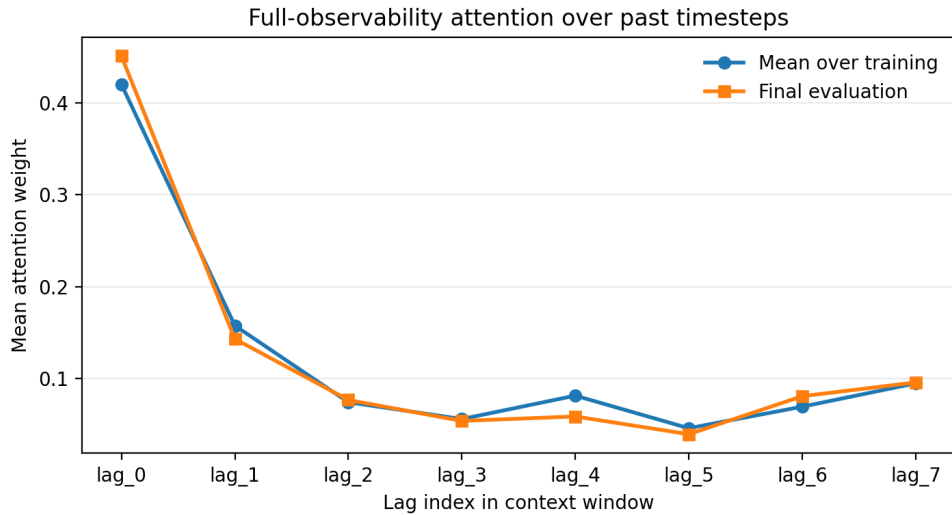


Figure 5: Mean attention weight by lag for the supplemental full-observation run. Both the training mean and final evaluation place the largest mass on the most recent token, while still assigning measurable mass across the context window.

7 Stability, Undertraining, and Checkpoint Choice

The finished suite does not mainly look undertrained. Most runs reached the requested one million steps, and many had their best checkpoint before the last evaluation. The stronger concern is instability after learning: 8 selected runs are flagged by the conservative late-deterioration threshold, with the clearest cases being p1 mlp s42, p1 mlp s43, p1 tr L8 s42, p2 hidden mlp s42, p2 hidden tr s42, p2 delay 30 mlp s42, p2 rlhf tr s42, p3 attn hidden s42. This is why the report keeps final and best returns separate.

The likely culprits are standard off-policy actor-critic failure modes: critic drift, extrapolation error from replay, bootstrapping noise, actor updates exploiting Q-function mistakes, and distribution shift as the learned policy changes. The POMDP wrappers add extra stress because noisy, delayed, or incomplete observations weaken the Markov assumption. Transformer policies add another sensitivity source: attention over a short history can help, but it also gives the actor a larger function class that may overfit critic errors.

Practical remedies are straightforward; save and report best-evaluation checkpoints in addition to final checkpoints, early-stop or select by validation return when compute is limited, run more seeds for high-variance settings, tune actor and critic learning rates separately, monitor Q-value scale and critic losses, increase evaluation frequency near peaks, and make target-policy updates more conservative in unstable settings. For this assignment I elected to report final return for strict end-of-run comparison, best return for learned capability, and the gap as a measure of run stability.

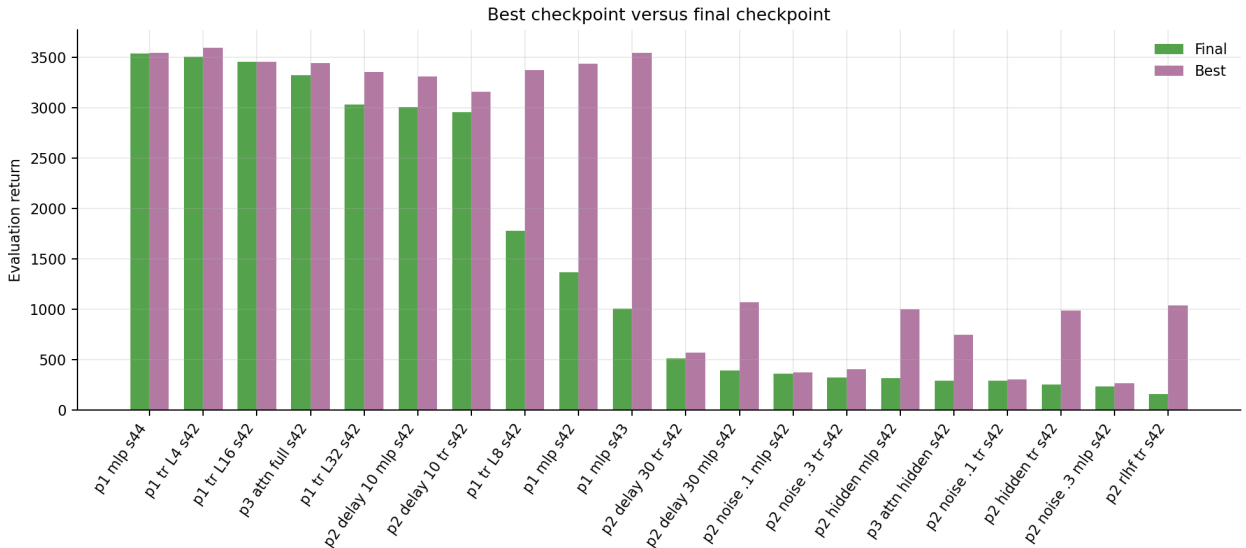


Figure 6: Final checkpoint return compared with best logged checkpoint return for every successful required run.

8 Hardware and Runtime Caveat

The final suite used both P100 and T4 hardware because of issues with Kaggle availability. Runtime therefore should not be read as an architecture-speed benchmark. T4 and P100 differ in GPU generation, available kernels, and, in this Kaggle setup, GPU count. The evaluation returns remain meaningful for algorithmic comparison because the environment, seeds, step budget, and training commands are the quantities that define the RL experiment, but hardware labels must travel with any wall-clock statement.

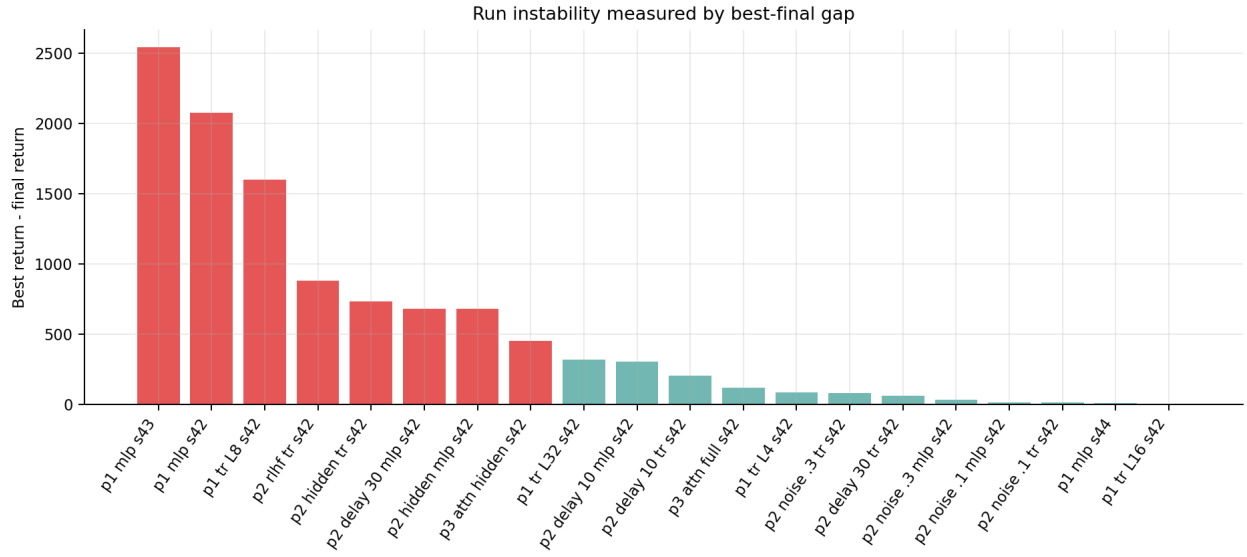


Figure 7: Best-final return gap. Larger bars indicate stronger late deterioration after a better earlier checkpoint.

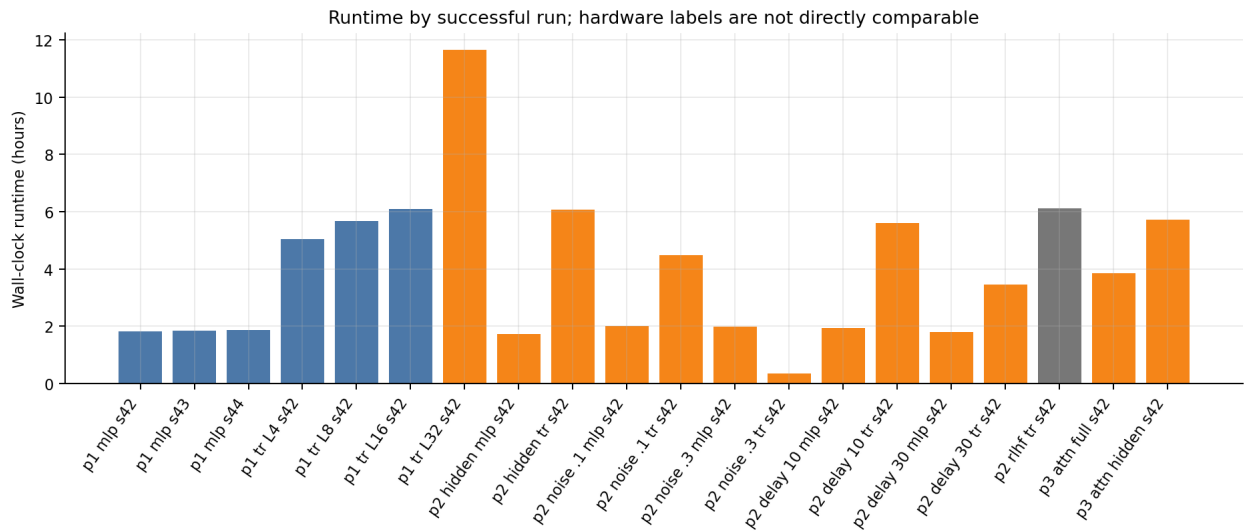


Figure 8: Wall-clock runtimes for successful runs. These values document execution cost, not a controlled hardware benchmark.

9 Conclusion

The required RL suite was completed with all 20 required final-tagged experiments present as successful W&B runs in the final seven-day window. The strongest result is that memory-based policies can be competitive and sometimes strong, especially in full-observation and delayed-reward settings. Speaking more conservatively, the conclusion is that Transformer memory is not automatically sufficient for POMDP robustness. Hidden velocities and noise remain difficult, and the learned-reward run is much weaker than ground-truth.

The most important reporting lesson is to separate three ideas that are easy to mix up. First, final evaluation return is the strict end-of-training result. Second, best evaluation return tells whether the method learned a good policy at any point. Third, the best-final gap measures stability. In this suite, the evidence points more toward instability and late deterioration than simple undertraining.

A Selected W&B Runs

- [rl_p1_mlp_seed42](#) (run id [orduqlzt](#), 2026-05-23T09:21:36Z; Part 1, MLP, Full; final 1364.2, best 3440.7, hardware Tesla P100-PCIE-16GB)
- [rl_p1_mlp_seed43](#) (run id [12tb4cex](#), 2026-05-23T11:10:50Z; Part 1, MLP, Full; final 1004.3, best 3543.1, hardware Tesla P100-PCIE-16GB)
- [rl_p1_mlp_seed44](#) (run id [xe1bvoqz](#), 2026-05-23T13:01:35Z; Part 1, MLP, Full; final 3540.2, best 3547.8, hardware Tesla P100-PCIE-16GB)
- [rl_p1_tr_L4_seed42](#) (run id [k4mmxrji](#), 2026-05-23T20:03:28Z; Part 1, Transformer, Full; final 3509.9, best 3596.1, hardware Tesla P100-PCIE-16GB)
- [rl_p1_tr_L8_seed42](#) (run id [jcs1nkci](#), 2026-05-24T01:05:56Z; Part 1, Transformer, Full; final 1778.8, best 3377.4, hardware Tesla P100-PCIE-16GB)
- [rl_p1_tr_L16_seed42](#) (run id [d6s0vtx6](#), 2026-05-24T10:58:11Z; Part 1, Transformer, Full; final 3459.6, best 3459.6, hardware Tesla P100-PCIE-16GB)
- [rl_p1_tr_L32_seed42](#) (run id [tc91exur](#), 2026-05-25T09:27:28Z; Part 1, Transformer, Full; final 3035.7, best 3353.2, hardware Tesla T4 x2)
- [rl_p2_hidden_vel_mlp_seed42](#) (run id [96m7x5eb](#), 2026-05-26T07:57:53Z; Part 2a, MLP, Hidden velocity; final 318.1, best 999.1, hardware Tesla T4 x2)
- [rl_p2_hidden_vel_tr_seed42](#) (run id [4hl8amlc](#), 2026-05-26T09:41:52Z; Part 2a, Transformer, Hidden velocity; final 253.0, best 986.3, hardware Tesla T4 x2)
- [rl_p2_noise01_mlp_seed42](#) (run id [lgmxbqai](#), 2026-05-26T15:45:40Z; Part 2b, MLP, Noisy; final 357.6, best 370.5, hardware Tesla T4 x2)
- [rl_p2_noise01_tr_seed42](#) (run id [3lt7il9g](#), 2026-05-27T16:57:39Z; Part 2b, Transformer, Noisy; final 288.4, best 300.5, hardware Tesla T4 x2)
- [rl_p2_noise03_mlp_seed42](#) (run id [anlzqstp](#), 2026-05-27T21:26:17Z; Part 2b, MLP, Noisy; final 234.3, best 266.7, hardware Tesla T4 x2)
- [rl_p2_noise03_tr_seed42](#) (run id [f4lc5xjg](#), 2026-05-28T05:24:55Z; Part 2b, Transformer, Noisy; final 325.0, best 405.5, hardware Tesla T4 x2)
- [rl_p2_delay10_mlp_seed42](#) (run id [523e0uha](#), 2026-05-28T05:46:42Z; Part 2c, MLP, Delayed reward; final 3007.5, best 3309.7, hardware Tesla T4 x2)
- [rl_p2_delay10_tr_seed42](#) (run id [hfiawzrg](#), 2026-05-28T07:42:38Z; Part 2c, Transformer, Delayed reward; final 2954.4, best 3158.5, hardware Tesla T4 x2)
- [rl_p2_delay30_mlp_seed42](#) (run id [15bzu83r](#), 2026-05-28T13:19:16Z; Part 2c, MLP, Delayed reward; final 390.4, best 1072.6, hardware Tesla T4 x2)
- [rl_p2_delay30_tr_seed42](#) (run id [kkdfxotj](#), 2026-05-28T17:58:13Z; Part 2c, Transformer, Delayed reward; final 511.8, best 571.9, hardware Tesla T4 x2)
- [rl_p2_rlhf_tr_seed42](#) (run id [rvckskwt](#), 2026-05-28T21:25:25Z; Part 2d, Transformer, Full; final 155.6, best 1035.3, hardware Unknown)
- [rl_p3_attn_support_full_seed42](#) (run id [78e2wcm0](#), 2026-05-29T07:34:06Z; Part 3, Transformer, Full; final 3321.9, best 3442.7, hardware Tesla T4 x2)
- [rl_p3_attn_support_hidden_seed42](#) (run id [lwsokb8n](#), 2026-05-29T11:25:30Z; Part 3, Transformer, Hidden velocity; final 293.6, best 745.3, hardware Tesla T4 x2)
- Supplemental attention-entropy source: [run jx1lmi8i](#) (2026-05-30T09:14:35Z; Part 3 full-observation entropy rerun; final 1397.6, best 3390.0, 100 logged entropy points)

B Reproducibility Notes

The selected-run CSV is stored at `rl/report/data/clean_rl_required_runs.csv`. The coverage audit is stored at `rl/report/data/coverage_audit.csv`, failed superseded attempts are stored at `rl/report/data/failed_superseded_runs.csv`, and raw W&B scalar metadata is stored at `rl/report/data/wandb_rl_runs_raw.csv`. The derived LaTeX tables are mirrored as CSV files under `rl/report/tables`. These files are included so final-return, best-return, stability-gap, hardware, and resume-status claims can be checked without relying only on the report prose.

References

- [1] Society for Artificial Intelligence and Deep Learning. *SAiDL Summer Assignment 2026*. Assignment specification, 2026.
- [2] Scott Fujimoto, Herke van Hoof, and David Meger. Addressing Function Approximation Error in Actor-Critic Methods. ICML, 2018.
- [3] Ashish Vaswani et al. Attention Is All You Need. NeurIPS, 2017.
- [4] Paul F. Christiano et al. Deep Reinforcement Learning from Human Preferences. NeurIPS, 2017.
- [5] Hila Chefer, Shir Gur, and Lior Wolf. Transformer Interpretability Beyond Attention Visualization. CVPR, 2021.
- [6] Joshua Achiam. Spinning Up in Deep Reinforcement Learning. 2018.
- [7] Michael Laskin et al. In-context Reinforcement Learning with Algorithm Distillation. 2022.
- [8] Maximilian Beck et al. xLSTM: Extended Long Short-Term Memory. 2024.